

Design Considerations

Autonomous Greenhouse

Carla Rubio Espiñeira

12 September 2026

Abstract

Contents

1. Tech stack	3
2. Goals of the system	3
3. Managed resources	3
4. Architectural pattern	4
4.1. Justification	4
5. Communication	4
6. Design considerations: Monitor	4
7. Design considerations: Knowledge	5
8. Design considerations: Analyzer	6
9. Design considerations: Planning Engine	7
10. Design considerations: Execution Engine	7
11. Conclusion	8
12. References	8

1. Tech stack

1. **Python 3**: language used across all components.
2. **Eclipse Mosquitto**: MQTT broker for the control plane.
3. **FastAPI + Uvicorn**: REST API for the knowledge base.
4. **Pydantic**: data validation for API payloads.
5. **Docker + Docker Compose**: containerization and orchestration of the whole pipeline.

2. Goals of the system

The main goal of the system is that the plants survive and grow. This can be achieved using the following sub-goals:

Goal	Description	Evaluation metric
Plant survival	The system must prevent metrics from reaching the alarm thresholds that could kill the plants.	e.g.: $(\text{isDay} == \text{True} \Rightarrow 12 \leq \text{Temp} \leq 32) \wedge (\text{isDay} == \text{False} \Rightarrow 10 \leq \text{Temp} \leq 22)$
Optimal growth conditions	The system should maintain all metrics within their ideal or secure ranges to promote plant health.	e.g.: $\text{isDay} == \text{True} \Rightarrow 18 < \text{Temp} < 26$
Energy and resource efficiency	When the system is not in an alarm state, it should minimize actuator usage (e.g., heaters, pumps) to save energy.	Minimization of active time for binary actuators (e.g., $\text{Heater} == \text{True}$)

3. Managed resources

The system includes the following sensors and actuators:

Metric	Monitored by	Managed by
Temperature	Temperature sensor	• Heater (increase) • Fan (decrease)
CO2	CO2 sensor	• Fan (decrease) • CO2 injector (increase)
Relative humidity	Hygrometer	• Sprinklers (increase) • Fan (decrease)
Ground humidity	Soil Moisture Sensor	• Water pump (increase) • Sprinklers (increase)
pH	pH Probe	• Acid dosing pump (decrease) • Base dosing pump (increase)
Conductivity (EC)	EC Sensor	• Nutrient dosing pump (increase) • Water pump (decrease)

4. Architectural pattern

The architectural pattern used is the MAPE-K loop.

4.1. Justification

5. Communication

The components communicate on two distinct ways:

Channel	Protocol	Use
Control	MQTT (greenhouse/...)	Actuator commands, planner plans, start / reset signals.
Data	HTTP REST (/short-term, /actuators, /logs...)	Synchronous reads and writes against the knowledge base.

The MAPE-K loop is structured as follows:

6. Design considerations: Monitor

The monitor function provides the mechanisms that collect, aggregate, filter, and report details collected from a managed resource.

1. What kinds of data and events are collected from which sources, sensors, or probes?

A simulator has been created to emulate the sensor and actuator components. It produces information with added randomization to imitate real-world fluctuations in the metrics, allowing evaluation of whether the autonomous system works under realistic conditions.

2. Are there common event formats?

The information produced by the simulator is emitted in JSON format into the monitor.

Inside this JSON, it is important to note that the simulation clock runs from 0 to 1439, where 0–719 corresponds to day and 720–1439 to night. This clock drives the day/night profile selection used throughout the system.

3. What is the sampling rate and is it fixed or varying?

The sampling rate is fixed at one minute.

4. Are the sampled sources fixed or do they change dynamically?

The sampled sources are fixed.

5. What are appropriate filters for the data streams?

Two filters are applied:

1. Every minute, the monitor retrieves the information from the last minute and checks whether there is an alarm or a dangerous metric. If so, the alarm protocol is activated.
2. Every 20 minutes, the system evaluates all the metrics of the greenhouse and activates the corresponding actuators. These actuators then self-regulate based on the metrics from the sensors and deactivate when needed.

7. Design considerations: Knowledge

A large portion of the knowledge base consists of monitored information.

1. How much information is needed for future reference?

Two types of memory are used:

1. **Short-term memory** (for the analyzer): the raw data accumulated over the 20-minute window. When the window ends, the short-term memory is reset, but a log entry is created that the system uses to determine whether the state is consistent or whether changes occurred. Approximately nine logs are retained so that the system can reason over the last approx. 3 h and detect emerging patterns.
2. **Long-term memory** (for future reference): a log of every important event, the time frame in which it occurred, related information, and the corrective action that was taken.

With the detailed level of reporting and logging available in modern software systems, it is important to monitor and store only the data that is genuinely useful to the control loop. If large amounts of log data are stored, performance may deteriorate because data is constantly monitored even when it has no relevance to the system. The knowledge component additionally contains:

1. The max/min tables for every metric used by the alarm protocols.
2. The priority hierarchy.
3. The active profile (day or night).
4. Both short- and long-term memory.
5. Symptom rules.

8. Design considerations: Analyzer

The analyzer provides mechanisms to correlate and model complex situations. It embodies the control model together with the planner, along with time-series forecasting and queuing models.

These mechanisms allow the autonomic manager to learn about the IT environment and to help predict future situations.

1. How are the collected data represented and stored?

The collected data is represented in a 20-minute log, in which all data is stored as JSON inside the knowledge base.

2. What are appropriate diagnosis methods to analyze the data?

Metric	Ideal measurement	Secure range	Alarm / Danger
Temperature (Day)	22 °C	18 °C – 26 °C	< 12 °C or > 32 °C
Temperature (Night)	16 °C	14 °C – 18 °C	< 10 °C or > 22 °C
Relative humidity	70%	60% – 80%	< 50% or > 85%
Ground humidity	40%	30% – 60%	< 20% or > 80%
CO2	800 ppm	400 – 1000 ppm	< 350 ppm or > 1500 ppm
Water / ground pH	6.0	5.5 – 6.5	< 5.0 or > 7.0
Conductivity (EC)	2.0 mS/cm	1.5 – 2.5 mS/cm	< 1.0 or > 3.5 mS/cm

3. How is the current state of the system assessed?

Two states are distinguished:

1. **Reference state:** the average of the last 3 hours, computed from the retained logs.
2. **Current state:** the new 20-minute window.

4. How much past state needs to be kept around?

The last nine evaluated states.

5. How are critical states achieved?

If a metric is dangerously out of range, the alarm protocol is activated and the system acts immediately, even outside the 20-minute evaluation window.

6. How are common symptoms recognized (e.g., symptoms database)?

Symptoms are derived from the alarm/danger column in the table above. Each case has its own symptom, for example, if the temperature is above 32 degrees and the soil humidity is below 20%, then the symptom would be “heating risk”.

9. Design considerations: Planning Engine

The planning engine provides mechanisms to construct the actions needed to achieve goals and objectives. It uses policy information to guide its work.

1. **How is the future state of the system inferred and how is a decision reached?** (via off-line simulation, quality-of-service (QoS) objectives, or utility goal functions.)

The future state corresponds to the next 20-minute window, evaluated from the data gathered. Decisions are reached if certain thresholds are crossed — for example, the target is to maintain the temperature between 18 °C and 26 °C, so if the forecast indicates it will rise above 26 °C, the blinds or a fan are activated to keep the plants cooler. All decisions follow a utility goal function so that the overall wellness of the system is maximized. Such metrics can be changed in the future to cater to a specific plant type or weather.

2. **What models and algorithms are used for trade-off analysis?**

In this system, energy cost is a secondary concern to plant survival. If the system is under alarm, it acts without consideration for energy costs. In all other instances, it regulates the metrics within the required range while minimizing energy consumption.

3. **What are the priorities for adaptation across multiple control loops and within a single control loop?**

The priority hierarchy is:

1. Extreme temperature or pH is fixed first, as it can kill the plant within minutes.
2. Soil moisture and relative humidity come next, as they are basic needs of the plant; the plant has a water reserve and can survive slightly longer without them.
3. CO₂ levels and conductivity adjustments come after, as they are less immediately dangerous for the plant.

4. **Under what conditions should adaptation be performed?**

Adaptation is triggered by changes in the normal metrics. Adaptation is also conditional on whether the system is in day or night mode, which is quantified by a simulation clock (so that experiments run faster than a real clock).

5. **How is head-room provided and system thrashing avoided, considering the timing of the required adaptations?**

To avoid system thrashing, hysteresis is used. For example, if the temperature should be between 18 °C and 26 °C, the heater is not stopped until the reading is further ahead of 18 °C — around 22–23 °C — after which a short cooldown of approximately 5 minutes is applied.

10. Design considerations: Execution Engine

The execution engine provides mechanisms to control the execution of a plan by updating the managed element dynamically.

1. **What are the managed elements and how can they be manipulated?**

The managed elements are the actuators, and all of them are manipulated through parameter tuning (e.g., switching the heater on or off). Each actuator is in a binary state of True or False, which determines whether it is active.

2. **Are changes of the system pre-computed, opportunistically assembled, composed, or generated?**

The system switches between known configurations, selecting pre-computed actions that are known to yield value. It does not create new combinations of known actions.

11. Conclusion

This document described the design of an autonomous greenhouse controller based on the MAPE-K loop, with an external and centralized architecture. The architecture separates reasoning (analyzer, planner) from actuation (executor), uses MQTT for the control plane and HTTP REST for the data plane, and maintains both short-term and long-term memory to support diagnosis and learning. Design decisions around filtering, priority, hysteresis, and the utility goal function were made explicit so that they can be revisited as the system evolves. Future work includes replacing the simulator with real hardware, adding actuator failure detection, and exposing the knowledge base through a monitoring interface.

12. References

1. Course material, “Autonomous Systems” (2025–2026), Università degli Studi dell’Aquila.